Requirements as the Oracle:

Generating REST API Tests from Vague Requirements Without Learning the Bugs They Should Catch

Team PreTrained

Sohail Memon	2025201075	Mohd Ahmad	2025201029
Kathan Patel	2025201039	Md Taufique Hussain	2025202007

1 Domain and Target User

Domain: Software engineering, specifically functional testing of REST (web) APIs.
Target users: Backend developers and QA engineers who have natural-language requirements (user stories, acceptance criteria) and an OpenAPI specification, and want automated tests that check the API *behaves* as required, not only that it does not crash.

2 Problem Statement

Automated REST API testing tools are effective at finding crashes. AutoRestTest won the SBFT 2026 REST tool competition, detecting about 2.7× more faults than the next tool [3, 2]. However, these tools use HTTP 5xx responses as their oracle, so a requirement violation that returns `200 OK` (e.g. a user deleting another user's data) goes undetected. Catching such bugs requires knowing the *expected* behaviour, which lives in natural-language requirements. RESTTestBench [1] measured how well LLMs turn requirements into executable tests: the best model catches up to 92% of planted bugs with precise requirements, but only 2–54% with vague ones, which is the realistic case in practice. A refinement loop (run the test, read the HTTP trace, regenerate) improves results, yet when the service under test is faulty the model "adapts its assertions to match the faulty behaviour" [1], so the refined test encodes the bug as correct. The paper measures this effect but proposes no remedy. Our problem is therefore to generate effective tests from vague requirements *and* to make refinement safe on possibly-buggy services.

3 Research Questions and Hypotheses

RQ1 (vague requirements). Given only a vague requirement and the OpenAPI specification, with no human input and no precise version at test time, can an LLM agent generate tests that catch requirement-violating bugs? How much of the vague-to-precise gap can it close?

RQ2 (safe refinement). Does restricting refinement (locked checks, hidden observed values on assertion failures, strengthen-only edits, an evidence-restricted judge) remove bug adaptation *without* increasing false alarms on the correct service?

RQ3 (component value). Which components (requirement-precision layer, retrieval, tool-based execution, restricted refinement, judge, fine-tuning) contribute measurably, and which variant of each works best? Can a fine-tuned 8B model approach a cheap API model?

Hypotheses. H1: deriving implied properties (round-trip, echo, ownership, before/after deltas, filter correctness) raises the bug-catching score on vague requirements well above the single-step baseline. **H2:** with restricted refinement, the score of tests refined against the buggy service is close to that of tests refined against the correct service (adaptation gap ≈ 0), with a false-alarm rate no higher than unrestricted refinement. **H3:** tool-based execution sharply reduces tests that fail on the correct service, which is the dominant failure of small models.

4 Why the Problem Matters

- Real requirements are vague. A test generator that works only on carefully written specifications has limited practical value.
- A test that silently accepts a bug is worse than no test, because it gives false confidence. Teams increasingly let LLMs write tests, so this failure mode will reach production code.

- Small models are cheap enough to run on every commit, but our re-analysis (Section 5.1) shows they currently fail on both problems.

5 What Is Missing in Existing Approaches

Approach	Reported performance	Gap for our problem
Crash-oriented fuzzers: EvoMaster, RESTler, AutoRestTest [2], LlamaRestTest [4], APIPilot [5]	AutoRestTest: 58.3% method coverage, 42 server errors on 12 services; SBFT 2026 winner	Oracle is 5xx/schema only; requirement violations returning 200 are invisible
Response-judging agents: LogiAgent [6], ARMeta [7]	LogiAgent: 234 logical issues, 66.2% accuracy; ARMeta: 57–79% true-positive rate	Judge sees the response before deciding, so it shares the same bias; many false alarms
Oracle generation: SATORI [8], MASTOR [9]	SATORI F1 74.3% (spec only); MASTOR 75.4% mutation score (needs Java source)	No requirements; MASTOR derives "truth" from the implementation
Requirement-based generation: APITestGenie [10], RESTestBench baselines [1]	APITestGenie: 69.3% valid scripts on 25 requirements; RESTestBench: 2–54% on vague requirements	Refinement freely rewrites expectations, so tests learn bugs; no remedy proposed

Table 1: State of the art (numbers as reported by each paper on its own setup; not directly comparable).

LLM-generated test oracles are known to capture the *actual* rather than the *expected* program behaviour [11], which is the mechanism behind bug adaptation.

5.1 Preliminary analysis of RESTestBench's released code and results

We inspected the authors' prompts and raw result files. Two findings motivate our design:

1. **The refinement prompts encourage adaptation.** The judge's few-shot example marks "the test expects `'Authentication failed'`, but the actual response contains `'Authentication error'`" as needing refinement, i.e. it teaches copying the observed value. The generator is told to "use the Test Execution Output and the HTTP Trace to enhance the test" and is never warned the service may be buggy. The judge's reasoning is not passed on (`USE_JUDGE_FEEDBACK_IN_REFINEMENT = False`), and there is no "API bug" outcome.
2. **Small models fail on mechanics and adapt to bugs.** For Llama 3.1 8B, excluding a failed run (0 valid tests), the share of planted bugs caught on vague requirements is 3.1% single-step, 12.3% after refinement on the correct service, and 3.5% after refinement on the buggy service: the entire refinement gain disappears. Across settings, 44–100% of its tests fail even on the correct service. (The paper's 13% figure for this model appears to include the failed run; without it, precise single-step is about 20%.)

6 Capabilities Used and Why the Problem Needs Each

Capability	Why the problem needs it	How we measure it
Tool calling	Small models get login flows, URLs, payloads and ID passing wrong; these are exact operations that tools can do deterministically from the spec	Valid-request rate, setup-error rate, tool-call argument validity, recovery after errors
Agentic	Testing is multi-step: create data, chain calls, observe state, decide whether a failure is the API's or the test's fault, and stop	Requirements completed end-to-end, steps per requirement, abstention precision
RAG	Specs are large; pasting the full spec (as RESTestBench does) overloads small models and wastes tokens	Recall@k, MRR, nDCG of retrieved operations; context tokens
Reasoning	Vague requirements must be expanded into concrete checks; the judge must reason about blame from evidence	Check recall and contradiction rate vs. precise requirements; judge verdict accuracy
Fine-tuning	Cost and the adaptation bias itself: train a small model to derive checks (SFT) and to report bugs instead of copying (DPO)	Bug-catching score, adaptation gap and cost vs. base 8B and an API model

Table 2: The five capabilities, the reason each is needed, and its metric.

7 Proposed Approach and Architecture

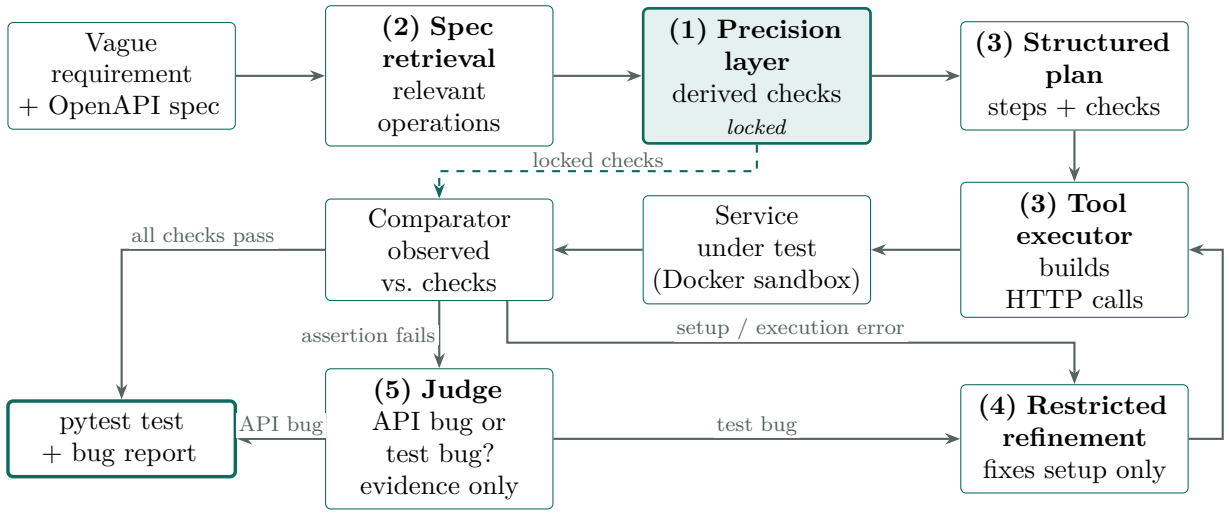


Figure 1: Proposed pipeline. Numbers refer to the components in the text. The shaded component is locked: execution feedback can fix how a test runs, but never the derived checks it compares against.

(1) **Requirement-precision layer.** Before any execution, the LLM expands the vague requirement into a set of atomic checks, guided by a small catalogue of implied properties (round-trip, echo, ownership/isolation, before/after deltas, filter soundness and completeness). Each check is tagged *meaning* (business rule, locked) or *format* (field names, envelopes, status codes the requirement does not fix; changeable only with spec evidence). The benchmark's precise requirements are used *only for evaluation*, never as input.

(2) **Spec retrieval (RAG).** Each OpenAPI operation (path, method, resolved schemas, description) is one chunk; the retriever returns the operations the requirement needs.

(3) **Structured plan and tool executor.** The LLM outputs a plan (actors, setup, action, checks), not raw code. Deterministic tools build requests from the spec (authentication flow, URLs, required fields, ID passing) and compile the plan to a pytest test.

(4) **Restricted refinement.** Execution errors (401/422 on setup, exceptions, missing spec-declared fields) are shown in full and may be fixed. Assertion failures are reported *without the observed value*. A guard allows only strengthening edits and flags any new expected value equal to a value from the last response (value copying).

(5) **Evidence-restricted judge.** On an assertion failure, the judge decides API_BUG or TEST_BUG, citing only requirement or spec text; "the server returned it" is never valid evidence. API_BUG ends the loop and keeps the test. Uncertain cases abstain.

(6) **Fine-tuning.** An 8B model (Llama 3.1 8B Instruct, for comparability with RESTestBench) trained with QLoRA: SFT on derived-check plans verified to catch planted bugs, then DPO preferring "report bug" over "adapt to the response". Leave-one-service-out training prevents leakage.

8 Expected Novelty

- The first remedy for the bug-adaptation effect identified by RESTestBench, with a mechanism-level explanation (prompt analysis and value-copy measurement).
- Separation of a test's *meaning* from its *format*, with explicit evidence rules for when each may change.
- Requirement-free-at-test-time expansion of vague requirements into implied, verifiable properties, evaluated against the benchmark's precise requirements.
- A component-by-component study (variants and metrics per component) of an agentic API test generator, including preference fine-tuning against adaptation.

9 Dataset and Evaluation Environment

- **RESTestBench** [1] (public): 3 Dockerised services (FastAPI/Python, NestJS Real-World/TypeScript, TodoApp/.NET); 106 requirements (63/29/14), each in a vague and a precise version; 228 hand-written, requirement-specific mutants (141/65/22); one reference test per requirement; scoring harness.
- **Optional extension:** 2–3 further APIs from EMB/WFD or RESTgym with ∼40 requirements and ∼80 mutants in RESTestBench's format, to test generalisation.
- **Sandbox:** all services run locally in Docker; tools may only call the service's base URL.

10 Baselines

All comparisons use the same LLM and budget.

B1. RESTestBench single-step generation (their prompt).

B2. RESTestBench refinement loop (their prompts, 3 rounds).

B3. *Prompt-fixed* refinement: their pipeline with only the prompts corrected (copying example removed, generator warned, judge feedback passed on, API_BUG outcome). This isolates "better prompting" from our structural changes.

B4. A response-judging agent in the style of LogiAgent [6].

B5. A crash/schema fuzzer (EvoMaster or Schemathesis), to show which requirement bugs crash oracles miss.

Reference points: published RESTestBench results for all 10 models, recomputed from the raw files with failed runs excluded.

11 Evaluation Plan and Metrics

11.1 End-to-end and safety metrics

- **Bug-catching score** (RESTestBench mutation score: mutants killed by tests that pass on the correct service), reported separately for vague and precise.
- **Gap closed** on vague requirements: (ours − baseline)/(precise upper bound − baseline).
- **Adaptation gap**: score of tests refined on the correct service minus score of tests refined on the buggy service.
- **False-alarm rate**: tests failing on the correct service.
- **Safety / unapproved actions:** expectation *tampering rate* (value copying and weakening edits), calls outside the sandbox allow-list, forbidden test edits (`pytest.skip`, removed assertions). Target: zero accepted.
- **Tool-call accuracy**: correct tool, valid arguments, successful recovery.
- **Cost**: tokens, $ and wall-clock time per requirement.

11.2 Component-level evaluation (variants × metric)

Following TA feedback, each component is justified by its own metric and compared across several variants.

Component	Variants	Component metric (ground truth)
Precision layer	none; direct rewrite; property-catalogue guided; self-consistency (5 drafts, majority); fine-tuned 8B	Check recall, contradiction rate, invented-check rate (atomic checks of the precise requirement)
Retrieval	full spec (no RAG); BM25; dense; hybrid (BM25+dense, RRF); hybrid + cross-encoder reranker; + LLM query rewriting	Recall@k, MRR, nDCG, context tokens (operations used by reference tests)
Generation format	raw pytest code; structured plan + tool compiler	Tests passing on correct service, setup-error rate, syntax validity
Refinement	none; unrestricted; prompt-fixed; hidden values; strengthen-only + copy detector; combined	Adaptation gap, value-copy rate, weakening rate, false alarms
Judge	none; RESTestBench judge; evidence-restricted; larger judge model	Verdict accuracy, API_BUG precision/recall (known: correct vs. mutated service)
Architecture	fixed pipeline; single agent with all tools (ReAct); multi-persona (precision, planner, executor, judge); multi-persona + shared blackboard	Tool-call accuracy, steps, cost, end-to-end score
Communication	tools in-process vs. via MCP server; agents via direct messages vs. A2A-style; content passed: full trace / summary / structured verdict	Latency, token overhead, failed calls; end-to-end score (content)
Model	Llama 3.1 8B base; +SFT; +SFT+DPO; GPT-5 Nano (reference)	Score, adaptation gap, cost

Table 3: Components, variants and component-level metrics.

11.3 Protocol

1. **Baseline reproduction:** run the RESTestBench harness and reproduce the published baselines before evaluating our components.

2. **Component sweep:** fix a default configuration and vary one component at a time on the FastAPI service (63 requirements), 1 run each.
3. **Finals:** best variants on all 3 services, vague and precise, 3 runs each, with paired Wilcoxon tests per requirement and effect sizes; full ablation removing one component at a time.
4. **Human validation:** any LLM-judged metric (e.g. check extraction) is validated against $\geq$100 human labels (Cohen's κ).
5. **Failure analysis:** taxonomy of surviving mutants and false alarms (wrong precondition, missing side-effect check, unreachable state, wrong blame), with qualitative examples.

12 Compute Plan

Item	Estimate
Models	Llama 3.1 8B Instruct for every agent. *Component sweep:* 4-bit quantised on the team's laptop GPUs. *Final runs:* fp16 on Kaggle, for comparability with RESTestBench's Llama 8B results. Optional: GPT-5 Nano as an API reference on a subset
Inference compute	*Local:* two laptops, RTX 5050 and RTX 4060 (8 GB VRAM each), running the 4-bit model and the Docker services together. *Kaggle:* 2× T4 (fp16 across both), ∼30 h/week per account, 4 team accounts. RESTestBench's Llama 8B refinement runs used ∼4–13M tokens each, mostly because the full spec is pasted into every call; our retrieval layer shortens prompts. Sweep (∼30 configurations × 1 run on FastAPI) plus finals (∼8 configurations × 3 services × 2 variants × 3 runs): ≈ **40–100 GPU-hours** in total (to be measured)
API budget	**$0** for the main experiments; optional GPT-5 Nano reference on a subset: ≈ **$10–30**
Fine-tuning	QLoRA SFT + DPO on the 8B model, ∼500–1,500 verified examples per fold, 3 folds: ≈ **20–30 h on Kaggle T4** (e.g. with Unsloth); the laptops are used only for small debugging runs (8 GB is tight for DPO)
Embeddings and reranker	Small open models (e.g. BGE-small, a MiniLM cross-encoder) on CPU or a single GPU; negligible cost
Infrastructure	Services under test and the benchmark harness run in Docker on the two laptops. Sweep runs execute fully on a laptop (model + services); for final runs the agent calls the Kaggle-hosted model through a tunnel (Kaggle does not support Docker). Mutation scoring is CPU-bound (rebuild/restart per mutant)

Table 4: Compute plan (estimates).

13 Expected Risks and Fallback Plan

Risk	Fallback
The adaptation gap is small for our models (the paper reports it only in a figure)	RQ1 (vague requirements) stands on its own; RQ2 is reported as measured, including a small or null effect
Locked checks cause false alarms when the derived check is wrong	Format checks relaxable with spec evidence; we report the false-alarm trade-off explicitly
4-bit quantisation (laptops) changes model behaviour	Final, reported runs use fp16 on Kaggle; we compare 4-bit and fp16 on a subset and report the difference
The 8B model cannot produce usable plans	Tool executor reduces what the model must get right; GPT-5 Nano as a fallback model
Too little fine-tuning data (106 requirements)	Multiple verified variants per requirement and paraphrases; few-shot small model as fallback, reported honestly
Evaluation leakage through the spec or precise requirements	Spec frozen from the correct service; precise requirements used only for scoring and leave-one-service-out training
A RESTestBench service is hard to run	Start with FastAPI (63 requirements, 141 mutants); report others as available
Kaggle quota or session limits (about 12 h) slow down experiments	Run the component sweep on the two laptops; spread final runs across the four Kaggle accounts; checkpoint long runs

Table 5: Risks and fallbacks.

References

[1] L. Kogler, S. Hangler, M. Ehrhart, B. Dornauer, R. Wuersching, P. Schrammel. *RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements.* EASE 2026. arXiv:2604.25862. Code: `https://github.com/casablancahotelsoftware/RESTestBench`

[2] M. Kim et al. *A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs (AutoRestTest).* ICSE 2025. arXiv:2411.07098.

[3] *SBFT Tool Competition 2026: REST League.* SBFT 2026. doi:10.1145/3786155.3795704; AutoRestTest report: arXiv:2607.01063.

[4] M. Kim et al. *LlamaRestTest: Effective REST API Testing with Small Language Models.* FSE 2025. arXiv:2501.08598.

[5] *REST API Testing with Verified LLM-Inferred Dependencies and Response-Driven Refinement (APIPilot).* arXiv:2608.17546, 2026.

[6] *LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents.* arXiv:2503.15079, 2025.

[7] *Multi-Agent LLM-based Metamorphic Testing for REST APIs (ARMeta).* arXiv:2605.28321, 2026.

[8] J. C. Alonso et al. *SATORI: Static Test Oracle Generation for REST APIs.* ASE 2025. arXiv:2508.16318.

[9] *MASTOR: A Multi-Agent Approach to Semantic Test Oracle Generation for RESTful APIs.* arXiv:2606.10465, 2026.

[10] *APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs.* arXiv:2604.02039, 2026.

[11] M. Konstantinou et al. *Do LLMs Generate Test Oracles that Capture the Actual or the Expected Program Behaviour?* arXiv:2410.21136, 2024.